EDUGENIE

Google Gemini Powered Learning Assistant

GENERATIVE AI PROJECT REPORT

A complete academic project with system design, modules, implementation, source code, testing and future enhancements.

Technology: Python, Streamlit, Google Gemini API, HTML/CSS concepts

Project Type: Generative AI / Educational Assistant

1. Abstract

EduGenie is a Generative AI based learning assistant designed to help students understand academic topics quickly and clearly. The application uses the Google Gemini model to generate explanations, summaries, study notes, practice questions and quiz content from a student's prompt.

The system provides a simple interface where a student enters a topic or question. The application sends the prompt to the Gemini API and displays the generated response. The project demonstrates how Generative AI can be integrated into an educational application using Python and Streamlit.

2. Problem Statement

Students often spend significant time searching multiple websites, reading long documents and converting difficult concepts into simple notes. A single learning interface that can explain a topic, summarize content and generate practice questions can make the learning process more convenient.

3. Objectives

• Create an AI-powered educational assistant.

• Integrate Google Gemini Generative AI into a student-friendly application.

• Generate simple explanations for difficult topics.

• Generate summaries, notes and practice questions.

• Provide an interactive chat-style learning experience.

• Demonstrate a practical Generative AI project suitable for academic presentation.

4. Existing System and Proposed System

Existing System

In a traditional learning workflow, students may use textbooks, search engines, video platforms and separate quiz resources. Information is distributed across multiple sources, and the student must manually organize it.

Proposed System

EduGenie combines several learning-support functions in one interface. The student enters a topic, selects a learning action, and receives AI-generated content. The application can be extended with authentication, history, file upload, voice input and other educational features.

5. Scope of the Project

• Topic explanation and question answering.

• Short and detailed summaries.

• Study notes and key points.

• Quiz and practice-question generation.

• Beginner-friendly explanations.

• Future integration with uploaded study materials.

6. Benefits

• Easy-to-use learning interface.

• Fast generation of study material.

• Supports personalized prompts.

• Reduces repetitive manual note preparation.

• Demonstrates real-world use of Generative AI.

7. System Requirements

Category	Requirement
Hardware	Computer/Laptop, 4 GB RAM or above recommended
Operating System	Windows / Linux / macOS
Language	Python 3.10+
Framework	Streamlit
AI Service	Google Gemini API
Internet	Required for API communication
Editor	VS Code / PyCharm / any Python IDE

8. Technology Stack

Python: Main programming language used for application logic.

Streamlit: Creates the interactive web interface without requiring a separate frontend framework.

Google Gemini API: Provides Generative AI capabilities for answering prompts and creating learning content.

Environment variables: Used to keep the API key outside the source code.

9. Project Folder Structure

```
EduGenie/
■
■■■ app.py
■■■ requirements.txt
■■■ README.md
■■■ .env
■■■ assets/
    ■■■ logo.txt
```

10. Main Application Code – app.py

The following is a complete starter implementation. Replace the API key with your own key through an environment variable; do not publish a real secret key in a public repository.

```
import os
import streamlit as st
from dotenv import load_dotenv
from google import genai

load_dotenv()

st.set_page_config(
    page_title="EduGenie - Gemini Learning Assistant",
    page_icon="■",
    layout="centered"
)

API_KEY = os.getenv("GEMINI_API_KEY")

st.title("■ EduGenie")
st.subheader("Google Gemini Powered Learning Assistant")

if not API_KEY:
    st.error("GEMINI_API_KEY is not configured.")
    st.stop()

client = genai.Client(api_key=API_KEY)

mode = st.selectbox(
    "Choose a learning mode",
    ["Explain Topic", "Summarize", "Create Notes", "Generate Quiz"]
)

topic = st.text_area(
    "Enter your topic or question",
    placeholder="Example: Explain photosynthesis in simple words"
)

if st.button("Generate"):
    if not topic.strip():
        st.warning("Please enter a topic or question.")
    else:
        instructions = {
            "Explain Topic":
                "Explain the topic clearly for a college student. "
                "Use headings and simple examples.",
            "Summarize":
                "Summarize the provided topic into concise study points.",
            "Create Notes":
                "Create structured study notes with headings, key points "
                "and important terms.",
            "Generate Quiz":
                "Create 5 useful quiz questions with answers."
        }

        prompt = instructions[mode] + "\n\nTopic:\n" + topic

        with st.spinner("EduGenie is generating your answer..."):
            response = client.models.generate_content(
                model="gemini-2.5-flash",
                contents=prompt
            )

        st.markdown("### ■ EduGenie Response")
        st.write(response.text)
```

11. requirements.txt

Create a file named requirements.txt and add the following packages:

```
streamlit
python-dotenv
google-genai
```

12. Environment Configuration

Create a .env file in the project folder. Keep this file private and never upload it to a public repository.

```
GEMINI_API_KEY=YOUR_API_KEY_HERE
```

13. Running the Project

1. Open the project folder in VS Code.
2. Create a Python virtual environment.
3. Install the packages:

   ```
   pip install -r requirements.txt
   ```

4. Add GEMINI_API_KEY to .env.
5. Start Streamlit:

   ```
   streamlit run app.py
   ```

6. Open the local URL shown by Streamlit in your browser.

14. How the System Works

→ Student enters a topic or question.

→ Student chooses a learning mode.

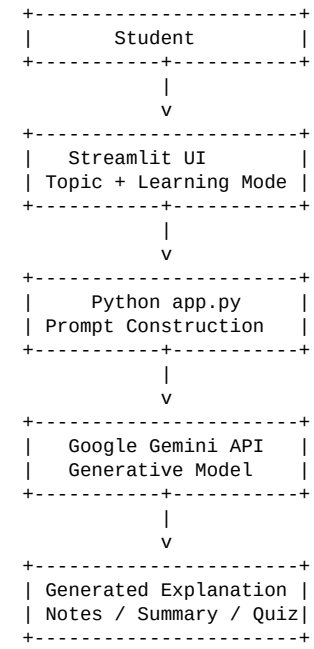
→ The Python application builds a structured prompt.

→ The prompt is sent to the Gemini model through the API.

→ Gemini generates the response.

→ Streamlit displays the response to the student.

15. System Architecture



16. Functional Modules

User Interface: Accepts topic/question and learning-mode selection.

Prompt Manager: Creates an instruction suited to the selected mode.

Gemini Service: Sends the prompt to the Generative AI model.

Response Display: Shows the generated learning content.

Configuration: Loads the API key securely from environment variables.

17. Sample Inputs

Explain Topic: “Explain the water cycle in simple words.”

Summarize: “Summarize the causes and effects of climate change.”

Create Notes: “Create study notes for data structures.”

Generate Quiz: “Generate a quiz about computer networks.”

18. Sample Output

For an input such as “Explain photosynthesis in simple words”, the application asks Gemini to produce a student-friendly explanation. The exact response can vary because it is generated by the AI model.

```
Sample structure:

PHOTOSYNTHESIS

1. Definition
Photosynthesis is the process by which green plants
make food using light energy.

2. Main Requirements
- Sunlight
- Water
- Carbon dioxide
- Chlorophyll

3. Importance
It supports plant growth and contributes oxygen to
the atmosphere.

4. Quick Revision
Plants + light + water + carbon dioxide -> food
```

19. Error Handling

```
try:
    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=prompt
    )
    st.write(response.text)
except Exception as error:
    st.error("Unable to generate a response.")
    st.caption(str(error))
```

In a production project, error messages should be logged safely without exposing secret keys or sensitive information.

20. Security Considerations

• Never hard-code an API key in source code.

• Use environment variables or a secret-management system.

• Do not publish .env files.

• Validate user input before sending it to external services.

• Avoid storing personal student information unless required and properly protected.

21. Testing

Test Case	Input	Expected Result
TC01	Explain a topic	AI explanation is displayed
TC02	Summarize content	Concise study summary
TC03	Create notes	Structured notes
TC04	Generate quiz	Questions and answers
TC05	Empty input	Warning message
TC06	Missing API key	Configuration error message

22. Advantages

• Simple interface suitable for student demonstrations.

• Multiple learning modes in one application.

• Fast generation of study content.

• Easy to extend with additional AI features.

• Good demonstration of Generative AI and API integration.

23. Limitations

• Generated answers can contain mistakes and should be checked against trusted study material.

• The application requires network access for API requests.

• API usage may have quotas, limits or costs depending on the selected service/account.

• The starter version does not include user authentication or a database.

24. Future Enhancements

• PDF and document upload with AI-based question answering.

• Student login and learning history.

• Voice input and text-to-speech.

• Automatic flashcard generation.

• Difficulty levels for quizzes.

• Progress dashboard and topic-wise performance.

• Multilingual learning support.

• Teacher/admin dashboard.

• Export generated notes as PDF.

25. Conclusion

EduGenie demonstrates a practical application of Generative AI in education. By combining Python, Streamlit and Google Gemini, the project provides an interactive learning assistant that can explain topics, summarize content, create notes and generate quizzes. The architecture is intentionally modular so that additional educational features can be added later.

26. Viva / Presentation Questions

Q: What is EduGenie?
A: An AI-powered learning assistant that uses Google Gemini to generate educational content.

Q: Why use Generative AI?
A: It can create natural-language explanations, summaries, questions and other learning content from prompts.

Q: What is Streamlit?
A: A Python framework used to build interactive data and AI web applications quickly.

Q: What is an API key?
A: A credential used by an application to authenticate with an API service.

Q: Why should the API key be hidden?
A: To prevent unauthorized use of the service and protect the account associated with the key.

Q: What is prompt engineering?
A: Designing clear instructions so the AI produces useful and relevant output.

27. Final Project Checklist

■ Project folder created.

■ app.py added.

■ requirements.txt added.

■ .env configured privately.

■ Dependencies installed.

■ Streamlit application tested.

■ Gemini response verified.

■ Screenshots can be added to the final college submission.

■ API key removed from any public source before submission.